

```
CREATE DATABASE IF NOT EXISTS `_93_b_2026`;
USE `_93_b_2026`;
```

```
-- =====
-- ЛАБОРАТОРНО 2: Създаване на структурата и данни
-- =====
```

```
-- -----
-- 1. Таблица: sports
-- -----
```

```
DROP TABLE IF EXISTS `sports`;
CREATE TABLE `sports` (
  `id` INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(50) NOT NULL
);
```

```
INSERT INTO `sports` (`name`) VALUES
('football'),
('volleyball'),
('basketball'),
('tennis');
```

```
-- -----
-- 2. Таблица: teachers
-- -----
```

```
DROP TABLE IF EXISTS `teachers`;
CREATE TABLE `teachers` (
  `id` INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(50) NOT NULL,
  `egn` VARCHAR(10) NOT NULL UNIQUE,
  `phone` VARCHAR(20) DEFAULT NULL
);
```

```
INSERT INTO `teachers` (`name`, `egn`, `phone`) VALUES
('Ivan', '1234567890', '0888767676'),
('Ivan Jr.', '3456789012', '0888545454'),
('Ivan II', '5612347890', '0888373737');
```

```
-- -----
-- 3. Таблица: students
-- -----
```

```
DROP TABLE IF EXISTS `students`;
CREATE TABLE `students` (
  `id` INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(50) NOT NULL,
  `egn` VARCHAR(10) NOT NULL UNIQUE,
  `address` VARCHAR(50) NOT NULL,
  `phone` VARCHAR(20) DEFAULT NULL,
  `class` VARCHAR(3) DEFAULT NULL
);
```

```
INSERT INTO `students` (`name`, `egn`, `address`, `phone`, `class`) VALUES
('Petko', '1234567890', 'ulica 5', '0888776667', '94'),
('Petko III', '4123567890', 'ulica 4', '0888232321', '93'),
```

```
('Pavel JR. MC', '3212567890', 'ulica 911', '0888141414', '92'),  
('Isen', '312231312', 'str. sth', '0899888884', '00');
```

```
-- -----  
-- 4. Таблица: sportGroups  
-- -----
```

```
CREATE TABLE `sportGroups` (  
  `id` INT AUTO_INCREMENT PRIMARY KEY,  
  `location` VARCHAR(50) NOT NULL,  
  `dayOfWeek` ENUM('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday',  
'Sunday') NOT NULL,  
  `hourOfTraining` TIME NOT NULL,  
  `sport_id` INT NOT NULL,  
  `coach_id` INT NOT NULL,  
  UNIQUE KEY(`location`, `dayOfWeek`, `hourOfTraining`),  
  CONSTRAINT `fk_sportGroups_sports` FOREIGN KEY(`sport_id`) REFERENCES `sports`(`id`),  
  CONSTRAINT `fk_sportGroups_teachers` FOREIGN KEY(`coach_id`) REFERENCES  
`teachers`(`id`)  
);
```

```
INSERT INTO `sportGroups` (`location`, `dayOfWeek`, `hourOfTraining`, `sport_id`,  
`coach_id`) VALUES  
('Sofia-Mladost4', 'Monday', '08:00:00', 1, 1),  
('Sofia-Mladost2', 'Friday', '15:00:00', 2, 3),  
('Sofia-Mladost1', 'Tuesday', '10:30:00', 3, 2),  
('Sofia-Mladost Tennis Center', 'Monday', '08:00:00', 4, 1);
```

```
-- -----  
-- 5. Таблица: student_sport  
-- -----
```

```
CREATE TABLE `student_sport` (  
  `student_id` INT NOT NULL,  
  `sportGroup_id` INT NOT NULL,  
  PRIMARY KEY(`student_id`, `sportGroup_id`),  
  CONSTRAINT `fk_ss_students` FOREIGN KEY(`student_id`) REFERENCES `students`(`id`),  
  CONSTRAINT `fk_ss_sportGroups` FOREIGN KEY(`sportGroup_id`) REFERENCES  
`sportGroups`(`id`)  
);
```

```
INSERT INTO `student_sport` (`student_id`, `sportGroup_id`) VALUES  
(1, 1), (2, 1), (3, 1), (2, 2), (3, 2), (2, 3), (1, 4), (2, 4), (3, 4);
```

```
-- -----  
-- 6. Таблица: programmers  
-- -----
```

```
DROP TABLE IF EXISTS `programmers`;  
CREATE TABLE `programmers` (  
  `id` INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  `name` VARCHAR(225) NOT NULL,  
  `address` VARCHAR(225) NOT NULL,  
  `startWorkingDate` DATE DEFAULT NULL,  
  `teamLead_id` INT DEFAULT NULL,  
  CONSTRAINT `fk_teamLead` FOREIGN KEY (`teamLead_id`) REFERENCES `programmers` (`id`)  
);
```

```

INSERT INTO `programmers` (`name`, `address`, `startWorkingDate`, `teamLead_id`) VALUES
('Ivan Ivanov', 'Sofia - Lozenets, bl. 12', '1999-05-25', NULL),
('Georgi Petkov Todorov', 'Sofia - Mladost 4, ul. 5', '1989-03-21', NULL),
('Teodor Ivanov Stoyanov', 'Sofia - Obelya, bl. 48', '2011-10-01', NULL),
('Maria Petrova', 'Sofia - Studentski Grad, bl. 7', '2015-06-15', 5),
('Dimitar Georgiev', 'Sofia - Lyulin, bl. 20', '2018-02-10', 5),
('Nikolay Dimitrov', 'Sofia - Center, ul. 12', '2020-09-05', 5),
('Elena Ivanova', 'Sofia - Vitosha, bl. 5', '2017-11-20', 5);

```

-- 7. Таблица: salarypayments

```

DROP TABLE IF EXISTS `salaryPayments`;
CREATE TABLE `salaryPayments` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `coach_id` INT NOT NULL,
  `MONTH` INT,
  `YEAR` YEAR,
  `salaryAmount` DOUBLE,
  `dateOfPayment` DATETIME NOT NULL,
  CONSTRAINT `fk_salary_coach` FOREIGN KEY (`coach_id`) REFERENCES `teachers`(`id`),
  UNIQUE KEY(`coach_id`, `MONTH`, `YEAR`)
);

```

-- 8. Таблица: taxesPayments

```

DROP TABLE IF EXISTS `taxesPayments`;
CREATE TABLE `taxesPayments` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `student_id` INT NOT NULL,
  `group_id` INT NOT NULL,
  `paymentAmount` DOUBLE NOT NULL,
  `MONTH` INT,
  `YEAR` YEAR,
  `dataOfPayment` DATETIME NOT NULL,
  CONSTRAINT `fk_taxes_student` FOREIGN KEY (`student_id`) REFERENCES `students`(`id`),
  CONSTRAINT `fk_taxes_group` FOREIGN KEY (`group_id`) REFERENCES `sportGroups`(`id`)
);

```

```

INSERT INTO `taxesPayments` (`student_id`, `group_id`, `paymentAmount`, `MONTH`, `YEAR`,
`dataOfPayment`) VALUES
(1,1, 200, 1, 2023, now()), (1,1, 200, 2, 2023, now()), (1,1, 200, 3, 2023, now()),
(1,1, 200, 4, 2023, now()), (1,1, 200, 5, 2023, now()), (1,1, 200, 6, 2023, now()),
(1,1, 200, 7, 2023, now()), (1,1, 200, 8, 2023, now()), (1,1, 200, 9, 2023, now()),
(1,1, 200, 10, 2023, now()), (1,1, 200, 11, 2023, now()), (1,1, 200, 12, 2023, now()),
(2,1, 250, 1, 2023, now()), (2,1, 250, 2, 2023, now()), (2,1, 250, 3, 2023, now()),
(2,1, 250, 4, 2023, now()), (2,1, 250, 5, 2023, now()), (2,1, 250, 6, 2023, now()),
(2,1, 250, 7, 2023, now()), (2,1, 250, 8, 2023, now()), (2,1, 250, 9, 2023, now()),
(2,1, 250, 10, 2023, now()), (2,1, 250, 11, 2023, now()), (2,1, 250, 12, 2023, now()),
(1,2, 200, 1, 2021, now()), (1,2, 200, 2, 2021, now()), (1,2, 200, 3, 2021, now()),
(4,2, 200, 1, 2021, now()), (4,2, 200, 2, 2021, now());

```

```

-- =====
-- ЛАБОРАТОРНО 3: Заявки, обновяване на данни и съхранени процедури
-- =====

-- Обновяване на името на учителя
UPDATE teachers SET name = "Ivan New" WHERE id = 1;

-- Прости селекти
SELECT * FROM students WHERE id = 2;
SELECT * FROM students WHERE id BETWEEN 1 AND 3;

-- Заявки с JOIN (Свързване на таблици)
SELECT sportGroups.location, sportGroups.dayOfWeek, sports.name
FROM sportGroups JOIN sports
ON sportGroups.sport_id = sports.id;

SELECT sportgroups.location, sportgroups.dayOfWeek, teachers.name
FROM sportgroups JOIN teachers
ON sportgroups.coach_id = teachers.id;

SELECT sports.name, teachers.name
FROM sports
JOIN sportGroups ON sportGroups.sport_id = sports.id
JOIN teachers ON teachers.id = sportGroups.coach_id;

-- OUTER JOINS
SELECT sportgroups.location, sportgroups.dayOfWeek, sportgroups.hourOfTraining,
sportgroups.sport_id, coaches.name
FROM sportgroups
LEFT OUTER JOIN teachers coaches
ON sportgroups.coach_id = coaches.id;

SELECT sportgroups.location, sportgroups.dayOfWeek, sportgroups.hourOfTraining,
sportgroups.sport_id, teachers.name
FROM sportgroups
RIGHT OUTER JOIN teachers
ON sportgroups.coach_id = teachers.id;

-- -----
-- изведете списък с имената на учениците класовете им, както и номера [id]
-- на групата в която тренират за всички ученици, които тренират в понеделник
-- от 8:00 и са при тренинг с определено име, но само за групите му по тенис.

SELECT students.name,
        students.class,
        sportGroups.id
FROM students

JOIN student_sport
ON students.id = student_sport.student_id

JOIN sportGroups
ON student_sport.sportGroup_id = sportGroups.id

```

```
JOIN teachers
    ON sportGroups.coach_id = teachers.id
```

```
JOIN sports
    ON sportGroups.sport_id = sports.id
```

```
WHERE sportGroups.dayOfWeek = 'Monday'
AND sportGroups.hourOfTraining = '08:00:00'
AND teachers.name = 'Ivan New'
AND sports.name = 'tennis';
```

```
-- -----
-- Задача 1
-- да се изведат всички спортове със спортни групи само от софия като
-- използваме само псевдоними на колони и таблици
```

```
SELECT s.name AS Sport,                -- sports -> s
       sg.location AS Location,
       sg.dayOfWeek AS DAY,
       sg.hourOfTraining AS TrainingHour
FROM sports s
JOIN sportGroups sg                    -- sportGroups -> sg
ON s.id = sg.sport_id
WHERE sg.location LIKE 'Sofia%';
```

```
-- =====
-- ЛАБОРАТОРНО 4: UNION, Подзаявки (IN), Self-JOIN
-- =====
```

```
-- Обединяване на имена и ЕГН-та на ученици и учители
```

```
SELECT name, egn FROM students
UNION
SELECT name, egn FROM teachers;
```

```
-- Пълен списък на групи, треньори и учители, които нямат група (LEFT JOIN + RIGHT JOIN)
```

```
SELECT sportgroups.location,
       sportgroups.dayOfWeek,
       teachers.name
FROM sportgroups
LEFT JOIN teachers
ON sportgroups.coach_id = teachers.id
UNION
```

```
-- учители които нямат група
```

```
SELECT sportgroups.location,
       sportgroups.dayOfWeek,
       teachers.name
FROM sportgroups
RIGHT JOIN teachers
ON sportgroups.coach_id = teachers.id
WHERE sportgroups.coach_id IS NULL;
```

```
-- Треньори чрез подзаявка (IN)
```

```
SELECT teachers.name AS CoachName, sports.name AS Sport
FROM teachers
```

```

JOIN sports
ON teachers.id IN (
    SELECT coach_id
    FROM sportGroups
    WHERE sportGroups.sport_id = sports.id
);

-- Извеждаме програмистите заедно с техните team leads (Self-JOIN)
SELECT prog.name AS programmersname,
       prog.address AS programmersaddress,
       teamLeads.name AS TeamLeadName
FROM programmers AS prog
LEFT JOIN programmers AS teamLeads
ON prog.teamLead_id = teamLeads.id;

-- Да се сформира двойка ученици на база спорт, който тренират
SELECT
    s.name AS Sport,
    st1.name AS Student1,
    st2.name AS Student2
FROM sports s
JOIN sportGroups sg ON sg.sport_id = s.id
JOIN student_sport ss1 ON ss1.sportGroup_id = sg.id
JOIN students st1 ON st1.id = ss1.student_id
JOIN student_sport ss2 ON ss2.sportGroup_id = sg.id
JOIN students st2 ON st2.id = ss2.student_id
WHERE st1.id < st2.id
ORDER BY s.name, st1.name, st2.name;

-- =====
-- ЛАБОРАТОРНО 5: Агрегатни функции, GROUP BY, HAVING и VIEWS
-- =====

-- Брой треньори
SELECT COUNT(coach_id) AS CountofSportgrwithCoaches FROM sportgroups;

-- Плащания
SELECT SUM(paymentAmount) FROM taxesPayments WHERE student_id = 2;
SELECT avg(paymentAmount) FROM taxesPayments WHERE group_id = 1;

SELECT group_id AS GrID, avg(paymentAmount) AS AVGOfpaymentperGroup
FROM taxesPayments
GROUP BY group_id;

-- Задача 1: Имената на децата и платените суми по месеци общо (Начин 1)
SELECT
    s.id AS StudentID,
    s.name AS StudentName,
    tp.MONTH AS MONTH,
    SUM(tp.paymentAmount) AS TotalPaid
FROM students s
JOIN taxesPayments tp ON s.id = tp.student_id
GROUP BY s.id, s.name, tp.MONTH

```

```
ORDER BY s.id, tp.MONTH;
```

```
-- Групиране с HAVING филтър (< 1000)
```

```
SELECT
```

```
    student_id,  
    students.name,  
    SUM(paymentAmount),  
    taxesPayments.MONTH
```

```
FROM taxesPayments
```

```
JOIN students ON taxesPayments.student_id = students.id
```

```
GROUP BY MONTH, student_id, students.name
```

```
HAVING SUM(paymentAmount) < 1000;
```

```
-- Създаване на изглед (VIEW) за график по футбол в понеделник
```

```
CREATE OR REPLACE VIEW football_training_schedule AS
```

```
SELECT
```

```
    st.name,  
    st.class,  
    sg.id AS group_id,  
    sg.dayOfWeek,  
    sg.hourOfTraining
```

```
FROM students st
```

```
JOIN student_sport ss ON st.id = ss.student_id
```

```
JOIN sportGroups sg ON ss.sportGroup_id = sg.id
```

```
JOIN sports sp ON sg.sport_id = sp.id
```

```
WHERE sp.name = 'Football'
```

```
    AND sg.dayOfWeek = 'Monday'
```

```
    AND sg.hourOfTraining = '08:00:00';
```

```
SELECT * FROM football_training_schedule;
```

```
-- =====  
-- ЛАБОРАТОРНО 6: Съхранени процедури (Stored Procedures) и променливи  
-- =====
```

```
-- Процедура 1: Извличане на всички групи със спортовете
```

```
DROP PROCEDURE IF EXISTS getAllSportGroupsWithSports;
```

```
DELIMITER |
```

```
CREATE PROCEDURE getAllSportGroupsWithSports()
```

```
BEGIN
```

```
    SELECT sg.location AS locationofGroup,  
           sg.dayOfWeek AS trainingDay,  
           sg.hourOfTraining AS trainingHour,  
           sp.name AS sportName
```

```
FROM sportgroups AS sg
```

```
JOIN sports AS sp
```

```
    ON sg.sport_id = sp.id;
```

```
END |
```

```
DELIMITER ;
```

```
CALL getAllSportGroupsWithSports();
```

```
-- Променливи
```

```

SET @coaches_name = 'Иван Тодоров Петров';
SELECT * FROM teachers WHERE name = @coaches_name;

-- Процедура 2: Входен параметър (IN)
DROP PROCEDURE IF EXISTS proc_in;
DELIMITER |
CREATE PROCEDURE proc_in (IN var VARCHAR(225))
BEGIN
    SET @coach_name = var;
END |
DELIMITER ;

CALL proc_in('Ilian Todorov');
SELECT * FROM teachers WHERE name = @coach_name;

-- Процедура 3: Входно-изходен параметър (INOUT)
DROP PROCEDURE IF EXISTS proc_in_withchange;
DELIMITER |
CREATE PROCEDURE proc_in_withchange (INOUT var VARCHAR(225))
BEGIN
    SET var = 'Ilian Todorov';
END |
DELIMITER ;

SET @testcoach_name = 'Ilian Todorov';
CALL proc_in_withchange(@testcoach_name);
SELECT @testcoach_name;

-- Процедура 4: Изходен параметър (OUT)
DROP PROCEDURE IF EXISTS out_Proc;
DELIMITER |
CREATE PROCEDURE out_Proc (OUT var VARCHAR(50))
BEGIN
    -- SELECT var; (only for test - премахнато за да не чупи MySQL)
    SET var = 'Ilian Todorov';
END |
DELIMITER ;

SET @testOutParam = 'Some name';
CALL out_Proc(@testOutParam);
SELECT @testOutParam;

-- Процедура 5: INOUT с друга логика
DROP PROCEDURE IF EXISTS proc_in_out;
DELIMITER |
CREATE PROCEDURE proc_in_out (INOUT var VARCHAR(225))
BEGIN
    -- select var; # only for test
    SET var = 'Иван Тодоров Петров';
END |
DELIMITER ;

SET @test_in_out_var = 'Some name';
CALL proc_in_out(@test_in_out_var);

```

-- Задача 1: Процедура, която показва децата, трениращи два или повече спорта

DROP PROCEDURE IF EXISTS students_with_two_sports;

DELIMITER |

CREATE PROCEDURE students_with_two_sports()

BEGIN

SELECT

st.id,

st.name,

GROUP_CONCAT(DISTINCT sp.name SEPARATOR ', ') AS sports

FROM students st

JOIN student_sport ss ON st.id = ss.student_id

JOIN sportGroups sg ON ss.sportGroup_id = sg.id

JOIN sports sp ON sg.sport_id = sp.id

GROUP BY st.id, st.name

HAVING COUNT(DISTINCT sg.sport_id) >= 2;

END |

DELIMITER ;

CALL students_with_two_sports();